IS883: Deploying Generative AI

Mohannad Elhamod

BOSTON UNIVERSITY

Boston University Questrom School of Business

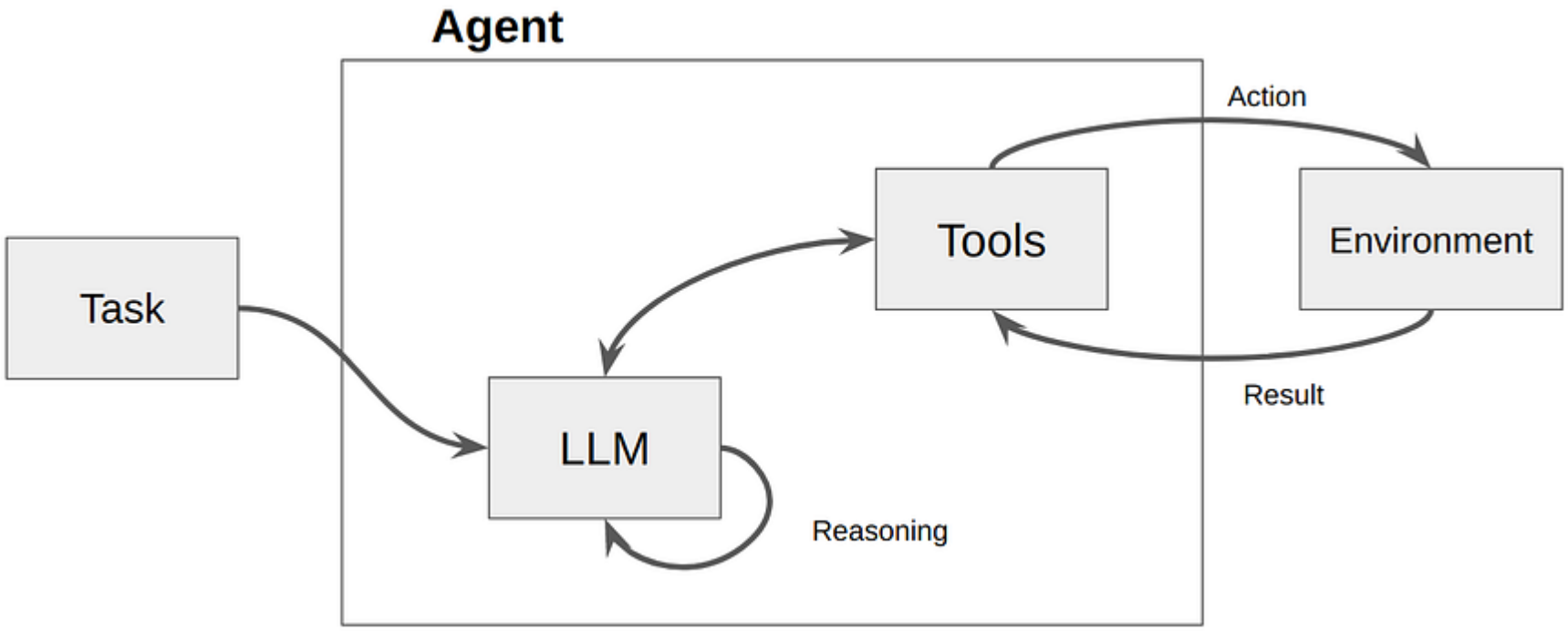
What is an Agent?

BOSTON UNIVERSITY

Demo

- Try some of the following prompts:
  - “Who is <*some obscure relative of yours*>*?*”
  - Calculate the product of the following numbers <*an array of large numbers*>

How Agents Work?



betterprogramming.pub/

What are Tools?

BOSTON UNIVERSITY

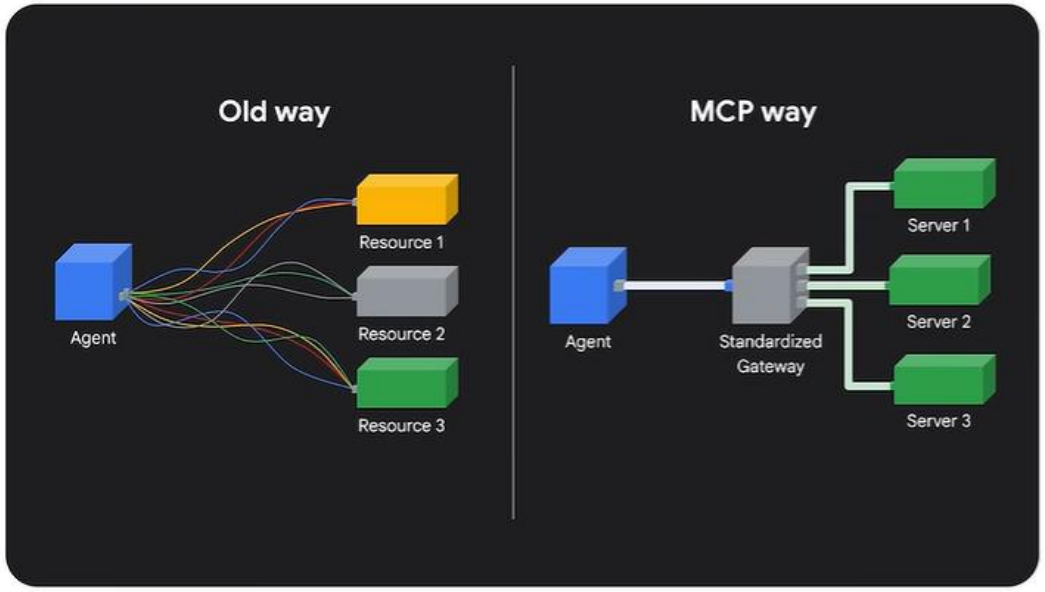
How Agents Work

- Examples of some tools.
- The model needs to "parse" the input text into arguments that can be used by the tool (e.g., API)

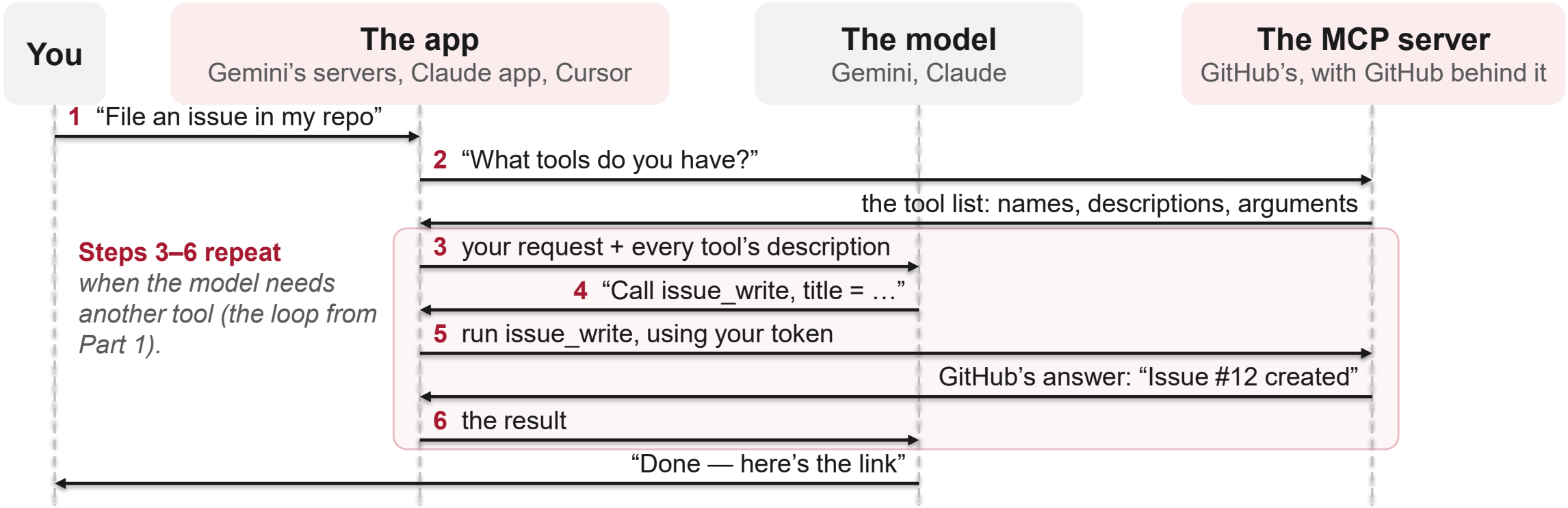


Tools in Practice: MCP

- MCP (Model Context Protocol): an open standard from Anthropic for connecting AI to external tools and data.
- mcp.so is a community directory of ready-made MCP servers (terminals, databases, agent memory, healthcare, and more).
  - Some provide MCP servers to use, while others provide open-source code to self-host.



How the model uses an MCP server



The model never talks to GitHub. The app makes every call, using your token.

Step 3 is where the token cost comes from: every tool's description is sent with every request (see next slide).

What MCP actually saves you

MORE TOOLS THAN YOU'D WRITE

1 tool vs. 80+ tools

Your function: file an issue. You wrote it; you maintain it.

GitHub's server: issues, pull requests, search, files… one address, kept current by GitHub.

How long would your team need to write and maintain all of these?

ZERO CODE TO PLUG IN

One line or one click

GitHub's server: plugs into Claude Code, VS Code, Cursor — no programming.

Your function: lives only in your notebook.

The people who use a tool are often not the people who could write it.

One limit: One small function, used in one app? Skip MCP — a plain function is simpler.

MCP adds a server, a network hop, and another party holding your token.
It also costs more tokens: every tool the server offers is described in every request, so a server with dozens of tools sends far more text than your one function — unless you limit the list, as allowed_tools does in the notebook.

Tool count: tool list in GitHub's MCP server README, retrieved 25 Oct 2026 — github.com/github/github-mcp-server

Who makes sure a model works with an MCP server?

Does the model maker train on each server, or does the server maker test every model?

THE MODEL MAKER

Google, Anthropic

Trains models to use tools in general: pick the right one, fill in the arguments, and not make up results. It does not train on your server. Smaller, cheaper models get this wrong more often.

THE APP IN THE MIDDLE

Gemini API, Claude app, Cursor

Connects to the server, fetches its tool list and hands each tool to the model in the model's normal format. The model never sees "MCP": a GitHub tool looks just like our compound_interest function.

THE SERVER MAKER

GitHub, MentorCruise

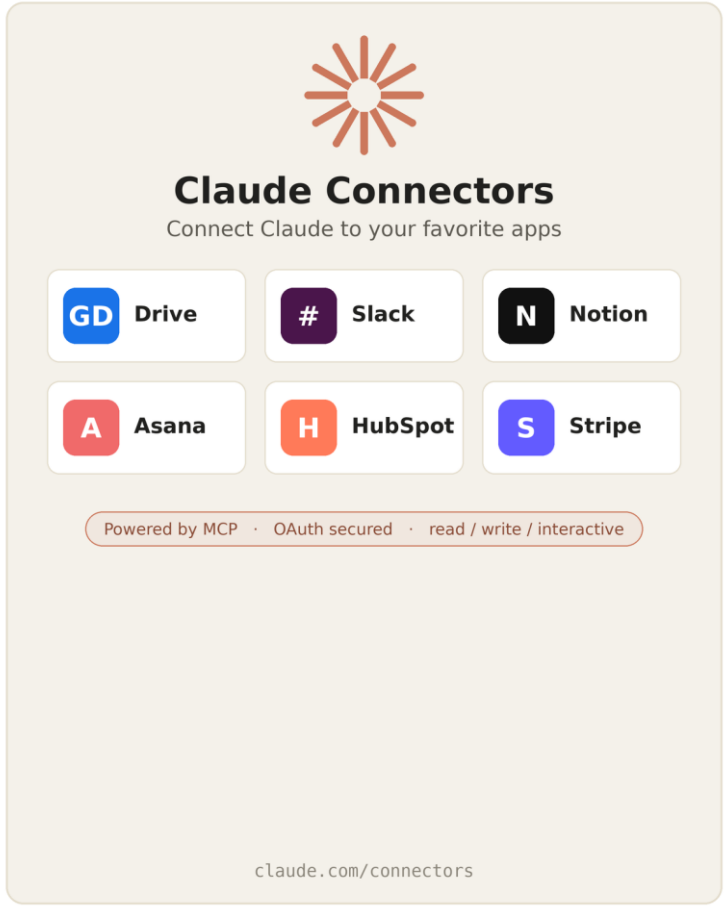
Writes the tool names, descriptions and arguments. They are the only thing the model reads, so they work like a prompt. It tests popular apps and models, but can't test them all.

Nobody guarantees your pairing. If you build with MCP, you test the model and server together, on your users' real requests — and re-test when you switch to a cheaper model.

Like websites and browsers: a shared standard, but nobody guarantees every site works in every browser.

Tools in Practice: Connectors

- Connectors = MCP made one-click, built right into Claude.
- Connect Claude to your favorite apps s, such as Google Drive, Slack, Notion, Stripe, etc.
- Claude can then read (and optionally write) your apps during a conversation



Tools in Practice: Skills

- A Skill = packaged expertise (instructions + a repeatable workflow) an agent can reuse. It is an advanced form of prompt engineering.
- Skill marketplaces let you install community skills into Claude in one click. (e.g., skillsmp.com. Take a look at finance-billing-ops)

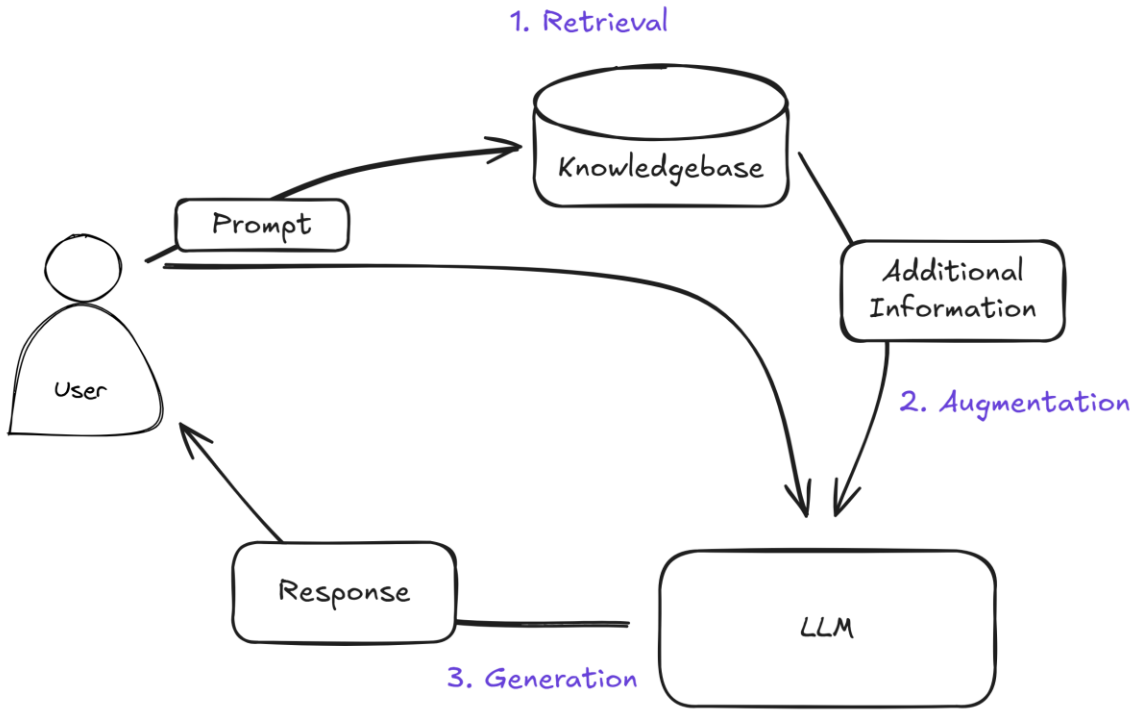
How to Access External knowledge?

BOSTON UNIVERSITY

Boston University Questrom School of Business

Incorporating Knowledge

- In its most basic form, the LLM only completes the prompt based on its internal parameters.
- Tools can provide more context for the LLM to make a better completion.
- More/better context could be provided through knowledge(e.g., text in files)



NVIDIA's Last Quarter

Prompt: "What was NVIDIA's total revenue last quarter?" — no internet, no search tool.

Without RAG

Answers from its parameters alone — the newest quarter it remembers. Write down the number and the quarter it names.

?

(run it live)

Fluent and confident — but is it the latest quarter?

With RAG

Retrieves the paragraph from NVIDIA's latest earnings release and answers from that text. Check it against the release.

?

(run it live)

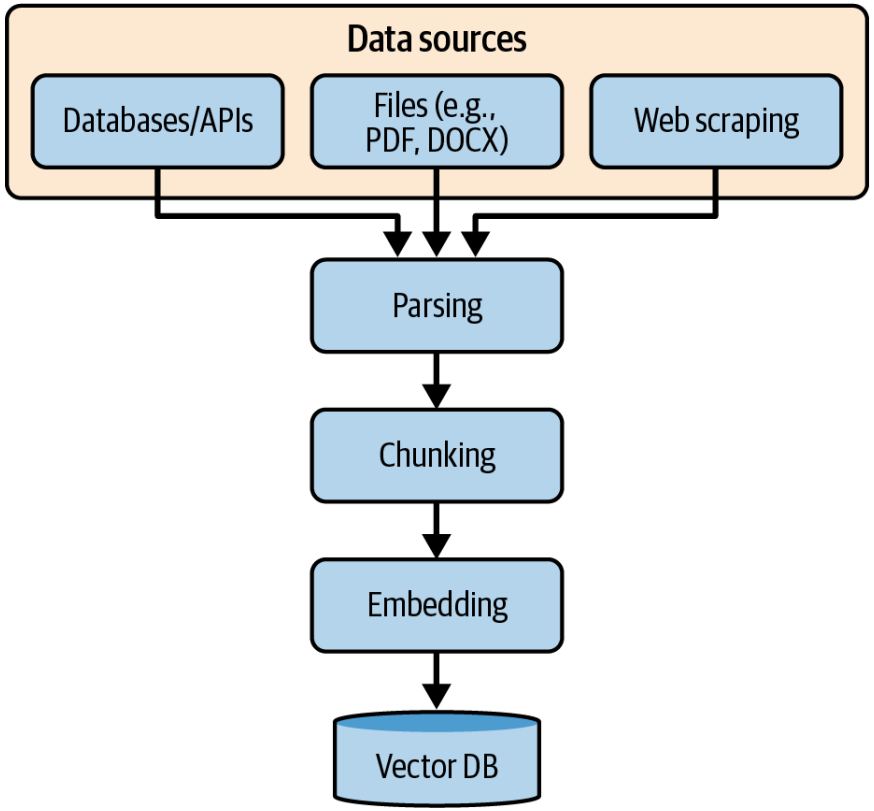
Answers from the retrieved text — and you can check the source.

Same model, same question. The only thing that changed is what was in the prompt.

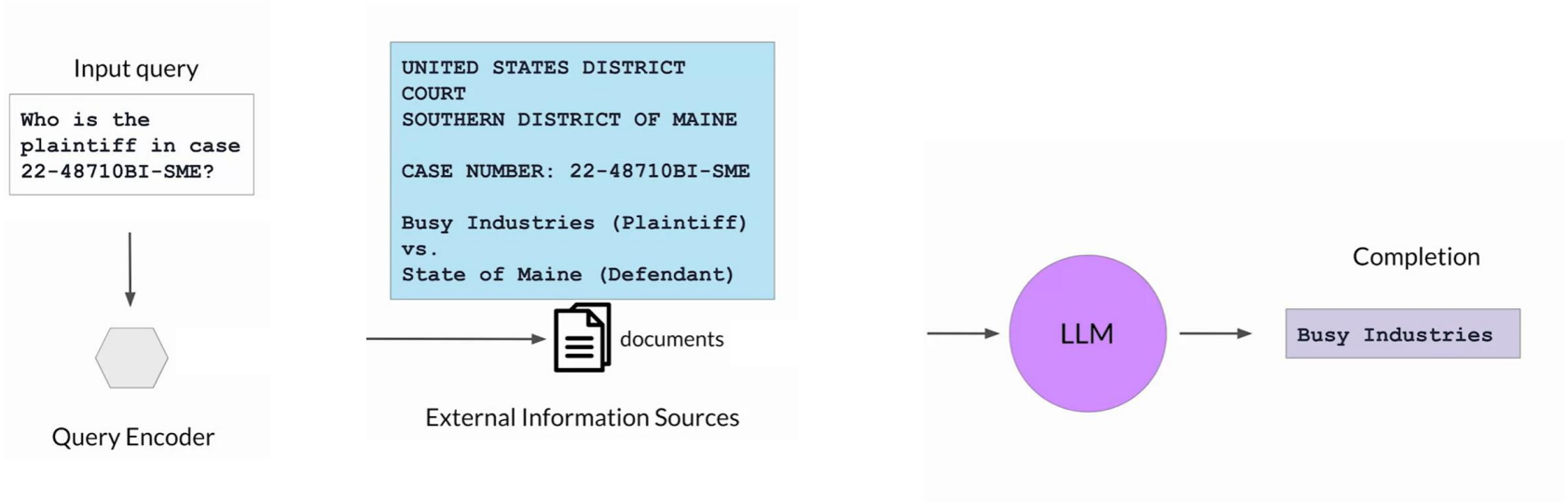
Retrieved text: NVIDIA's latest quarterly results press release (nvidianews.nvidia.com).

Knowledge is Overwhelming!

- Knowledge may contain huge amounts of data, most of which may not be relevant.
- The model needs to find the *chunks* that are relevant to the prompt in question.
- Finding the appropriate chunk(s):
  - Storing all chunks as embeddings in a Vector database
  - Extracting the chunks with high similarity to the prompt (as measured by cosine similarity)
- This is called **RAG**

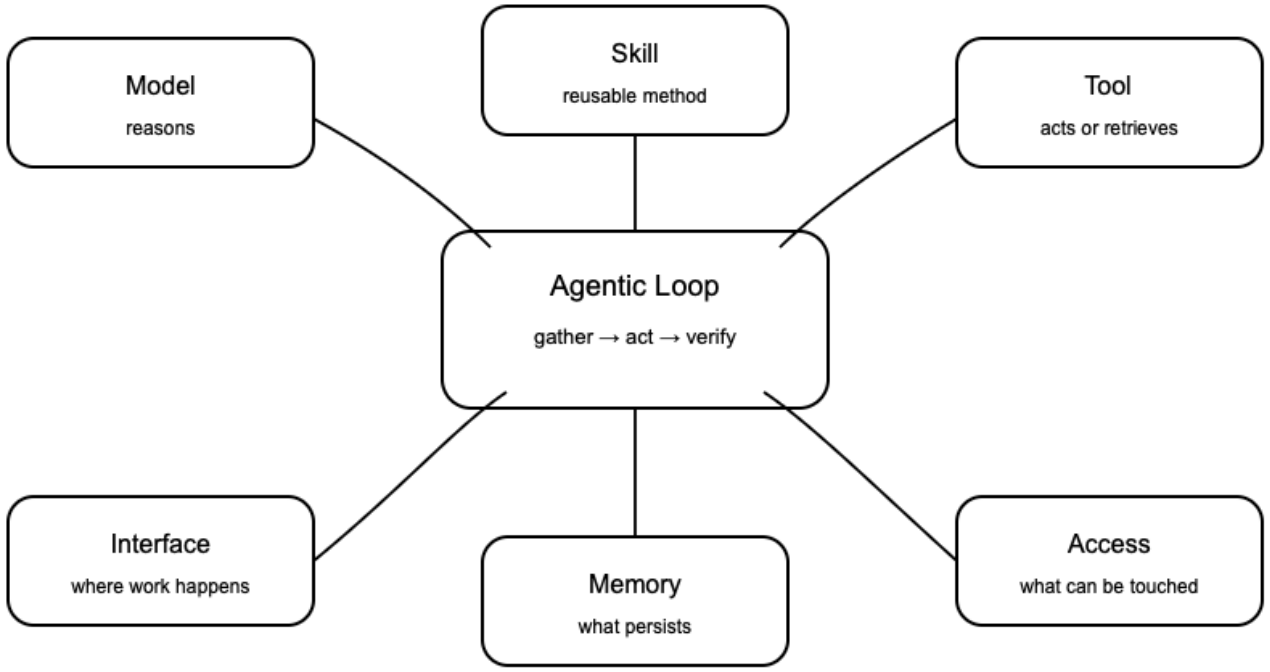


Example



coursera.org

The Big Picture



The model does not work alone. The surrounding layers shape what agentic work can do.